PRE-LABORATORY 2

COMMON ADVANCED HAND-GESTURE COMPUTATIONAL FOUNDATION

One identical Pre-Lab for all 10 students (V01–V10). Individual variants apply only to the Actual Laboratory 2.

1. Objective and Starting Code

Extend the instructor-provided OpenCV + CVZone DragRect application into a reliable, computation-driven, pinch-to-grab interaction engine. This is an implementation activity, not a proposal. Every student must implement all requirements below before beginning their assigned Lab 2 variant.

2. Required Functional Implementation

A. Stable hand tracking

Retain camera-index fallback, mirrored live feed, index-tip tracking (landmark 8), and c to create a rectangle. Handle temporary loss of hand and cleanly release webcam and windows on q.

B. Explicit selection and drag state machine

Implement IDLE → HOVER → GRABBED → DRAGGING → RELEASED, plus INVALID/COLLISION where applicable. Use thumb (landmark 4)–index (landmark 8) pinch to grab; a hover alone must not move objects. Only one object may be grabbed at a time; resolve overlaps by topmost or nearest object.

C. Rectangle geometry and hit-testing

Compute left/right/top/bottom bounds from center and size; determine whether the cursor is inside and compute cursor-to-center Euclidean distance. Enforce visible workspace bounds.

D. Motion measurement

Compute frame-to-frame displacement, Δt, velocity components, speed, and acceleration. Guard Δt against zero; smooth noisy motion with a moving average or exponential filter.

E. Pinch recognition

Calculate thumb–index Euclidean distance and implement separate grab/release thresholds (hysteresis) to avoid flickering. Display pinch distance and selected state.

F. Collision analysis

For two axis-aligned rectangles, compute intersection width, height, area, and percentage overlap relative to the smaller rectangle. Display collision warnings and prevent or visually flag invalid placements.

G. Advanced add-on behaviors

Implement both (1) snap-to-grid with configurable cell size and snap error and (2) dwell-to-lock with a visible timer; these must change object behavior, not only show text.

H. Debug and visual feedback

Live panel: selected object ID, state, bounds, cursor distance, displacement, speed, acceleration, pinch distance, collision area/percentage, grid cell, snap error, and dwell timer. Show state-dependent colors and a selection outline.

I. Error handling

Handle no camera, no hand, incomplete landmarks, off-screen cursor, overlapping selection, invalid drop, duplicate input, and large frame-time gaps without crashing.

3. Minimum Mathematical Computations

- Bounds: $L=cx-w/2$; $R=cx+w/2$; $T=cy-h/2$; $B=cy+h/2$.
- Distance: $d=\sqrt{[(x_2-x_1)^2+(y_2-y_1)^2]}$.
- Displacement and velocity: $\Delta p=p_t-p_{t-1}$; $v=\Delta p/\Delta t$; speed$=\sqrt{(vx^2+vy^2)}$.
- Acceleration: $a=(v_t-v_{t-1})/\Delta t$.
- Pinch distance: $d_p=\sqrt{[(x_8-x_4)^2+(y_8-y_4)^2]}$.
- Intersection: $w_i=\max(0,\min(R_1,R_2)-\max(L_1,L_2))$; h_i similarly; $A_i=w_ih_i$; overlap%$=100A_i/\min(A_1,A_2)$.
- Grid snap: $x_s=g\cdot\text{round}(x/g)$, $y_s=g\cdot\text{round}(y/g)$; snap error$=\sqrt{[(x-x_s)^2+(y-y_s)^2]}$.
- Dwell: t_dwell=t_now−t_enter; lock only if stable and threshold reached.

4. Required Manual Verification

Show five fully worked runtime examples with actual captured coordinates and substituted values: (1) rectangle bounds/hit-test, (2) cursor distance, (3) displacement/velocity/acceleration, (4) pinch thresholds, and (5) collision overlap and grid snap. Explain differences caused by rounding and frame timing.

5. Functional Test Cases

Provide at least 10 cases with test ID, input/precondition, expected result, actual result, Pass/Fail, and screenshot/video timestamp. Include valid drag, release, overlap, grid snap, dwell lock, hand loss, boundary, camera failure, overlapping objects, and q exit.

6. Required Submission

- Separate Pre-Lab 2 document (PDF or DOCX): objectives, architecture, flowchart, state-transition table, equations, five manual solutions, ten test cases, screenshots, reflections.
- Runnable Python source code and required dependency list; document how to run from VS Code.
- Video 1: starter code running. Video 2: complete common Pre-Lab implementation showing computations and both add-on behaviors.
- The finished Pre-Lab code must remain operational and be integrated into the individually assigned Actual Lab 2.

7. Grading Rubric (100 Points)

Criteria	Points
Starter integration and robust tracking	10
Computations and correct formulas	25
State machine and gesture interaction	20
Grid snap, dwell lock, collision behavior	20

Manual verification	10
Testing and edge cases	10
Documentation and videos	5